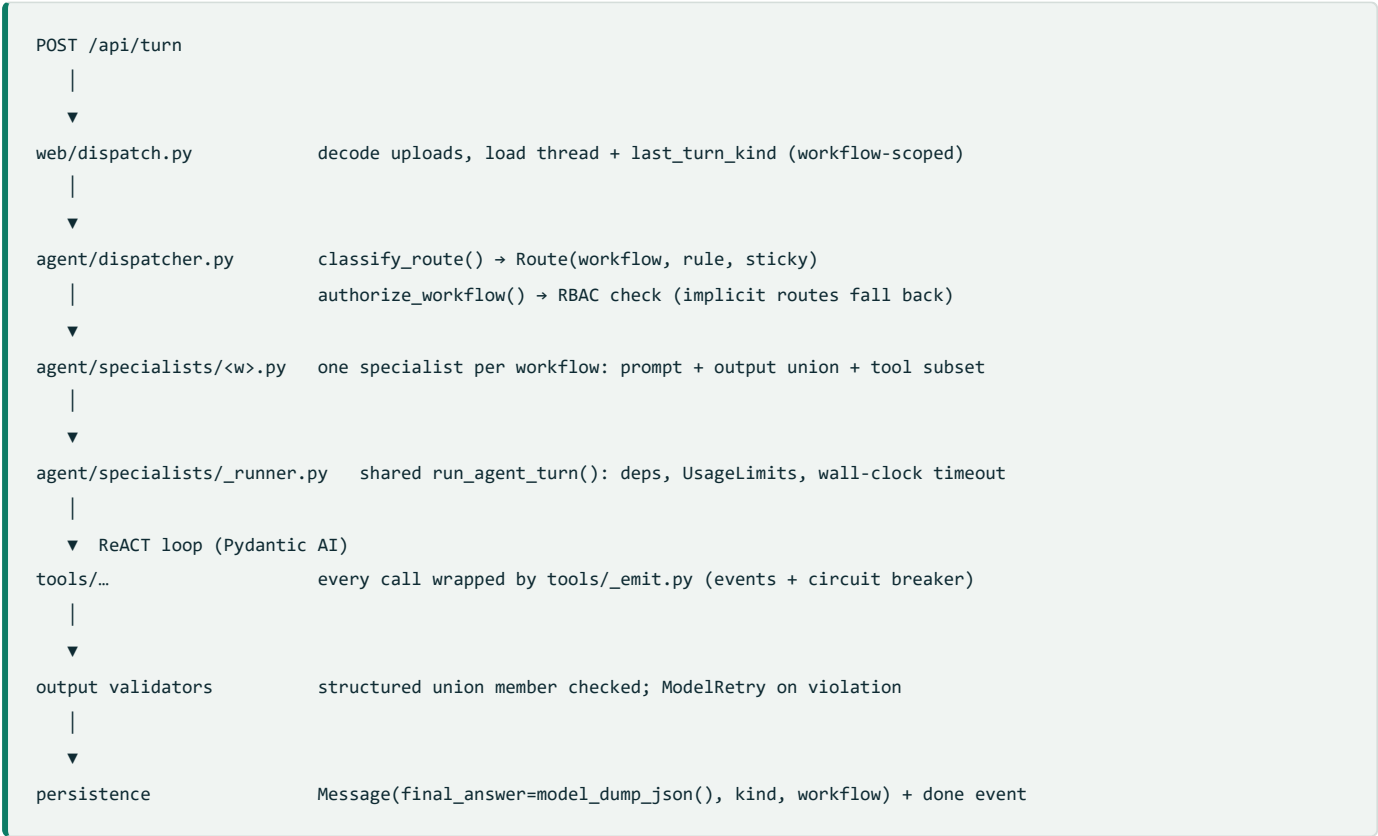


Agentic Architecture & Design Practices

A concise tour of how CRA is built: routing and intent analysis, the ReACT loop and its guardrails, the anti-hallucination framework, deterministic Python workflows, FastAPI/SQLAlchemy conventions, security, and regulatory compliance. Forward-looking items are marked TODO and tracked in `trial-readiness.md` / `roadmap.md`.

Companion docs: `../architecture.md` (target system topology), `agent-loop-review.md` (the review that produced the current loop design).

1. One turn, end to end



Design rule: **the LLM decides content; Python decides control flow**. Routing, authorization, stage gates, budgets, timeouts, and validation are all deterministic code the model cannot talk its way around.

2. Intent analysis & routing

`agent/dispatcher.py::classify_route()` is a deterministic rule ladder — cheap, auditable, and testable without a model. Each decision is a frozen `Route(workflow, rule, sticky)`; the `rule` is persisted in the turn's `done` event so routing behaviour is observable in production.

Order	Rule	What it does
1	slash	/meta , /general , ... force a workflow (403 if unauthorized)
2	continuation	"PICO confirmed" etc. — scoped to the <i>pinned</i> workflow only
3	handoff	_HANDOFF_SEEDS : "Draft a manuscript from this meta-analysis" jumps workflows
4	definitional	"What is a forest plot?" detours to general_qa without re-pinning (sticky=False)
5	pinned	thread already in a workflow → stay in it
6	keyword	keyword match selects a workflow
7	default	general_qa
—	llm_fallback	only when rule = default : one cheap structured Haiku call (fallback_classifier.py), Literal-typed output, hard timeout, fail-open to general_qa
—	auth_fallback	implicitly-inferred routes the user lacks permission for degrade to general_qa instead of erroring

Two properties worth copying elsewhere:

- **Sticky vs detour.** Only routes meant to change thread state re-pin the thread. Mid-workflow side questions answer and return; the `Message.workflow` column scopes `last_turn_kind` so detours can never corrupt a workflow's stage machine.
- **LLM as fallback, not front door.** The model classifier runs only when every deterministic rule has passed, is capped at 2 requests and a timeout, and any failure degrades to the safe default. Routing never *depends* on a model call succeeding.

3. The ReACT loop and its guardrails

Specialists are Pydantic AI `Agent`s: the model reasons, picks a tool, observes the result, repeats, then emits a **structured output** (a discriminated union — e.g. `question` | `pico_draft` | `final_report`). The union's `kind` doubles as the workflow stage marker (`last_turn_kind`).

`_runner.py::run_agent_turn()` is the single execution path for all 17 specialists and enforces the hard bounds:

- `UsageLimits(request_limit, tool_calls_limit)` — per-turn model-request and tool-call budgets (general_qa: 40 tool calls; global `max_model_requests=100`).
- `asyncio.wait_for(...)` wall-clock timeout (`agent_timeout_seconds=120`) — a stuck tool cannot hold a turn open.
- One shared `AgentDeps` per turn carrying breaker state, uploads, and stage context.

Circuit breaker (`tools/_emit.py`) — three escalating layers so the agent can never burn a turn repeating itself:

1. **Repeat-call cache.** An identical (tool, canonicalized-args) call is served from cache with a `[REPEATED CALL]` marker and a "do not repeat — change your approach" nudge. The observation *changes*, which is what actually breaks model loops.
2. **Per-tool disable** after 3 failures of the same tool.
3. **Global soft-disable** after 5 total failures: all tools switch off and the model is told to produce a best-effort answer from what it has. `ToolErrorBudgetExceeded` (a hard 502) fires only if it *keeps* calling tools after that — graceful degradation first, abort as backstop.

Retry hygiene. Transient-failure retries live at the right layer: `config/rate_limit.py` retries 429/5xx/transport errors with backoff and `Retry-After`; `ModelRetry` handles semantic tool errors the model can fix by changing arguments; output-validation retries are bounded. Nothing retries blindly at two layers at once.

4. "Is the assistant ready to answer?" — staged validation

Readiness is enforced structurally, not by prompting alone:

- **Tool gating by stage** (`prepare_tools`). In `meta_analysis`, `search_papers` / `fetch_pmc_fulltext` / `sandbox_exec` are *invisible* until the conversation reaches the right `last_turn_kind`. The model cannot skip PICO confirmation and jump to synthesis, because the tools to do so don't exist yet. Upload-gated tools work the same way (`gate_attachment_tools` hides `describe_image` unless the user actually attached an image).

- **Output unions as state machines.** A specialist can only return one of its declared output types; the persisted `kind` becomes the next turn's gate input. There is no free-text "final answer" — everything round-trips through a schema.
- **Output validators** run after the model claims to be done and raise `ModelRetry` with a specific instruction when the answer isn't acceptable (see §5). A final answer that fails validation is not a final answer.
- **Host-side acceptance checks.** Where "good enough" is measurable, it is measured in Python, not self-assessed: lay summaries are scored with a host-side Flesch-Kincaid implementation (`services/readability.py`) and the turn retries until the target grade level is met.

5. Anti-hallucination framework

The strictest constraint in the codebase, added after real fabrication incidents. The principle: **every clinical claim must trace to an artefact the system itself produced or retrieved.**

- **Provenance-only citations.** PMIDs may only come from `search_papers` results; registration_drafter **never fabricates NCT IDs**; general_qa's `_reject_clinical_synthesis` validator rejects answers quoting effect sizes, PMIDs, or guideline citations from training data (red-flag regexes deliberately match same-line only — `[ \t:]*` — so numbered lists across paragraph breaks don't false-positive).
- `derived_from` **lineage.** In CSR drafting and trial_stats, every numeric claim carries a `derived_from` field naming the dataset/analysis that produced it; validators reject unattributed counts.
- **Computed, not asserted.** GRADE certainty is a Pydantic `computed_field` (`compute_certainty()`) — the model supplies domain judgments (risk of bias, imprecision, ...) and `code` derives the rating. Same pattern: readability scores, meta-analysis statistics (sandbox scipy/statsmodels, never model arithmetic), sample-size calculations.
- **Never relax without sign-off.** These guardrails are load-bearing; loosening any of them requires explicit maintainer approval (see `CLAUDE.md`).

6. Deterministic workflows: push work into Python

Anything mechanical, auditable, or regulator-facing is plain Python — the model orchestrates but does not compute:

- `sandbox_exec` — Docker-isolated Python (network disabled, read-only script/input mounts, RW output mount, CPU/memory/time limits, guaranteed container removal on success/error/cancel). All statistics run here: meta-analysis pooling, K-M/Cox/MMRM, NMA/SUCRA, funnel/Egger.
- **Host-side pure functions** — Flesch-Kincaid, GRADE certainty, sample size, Bedrock pricing/cost rollup, CDISC dataset derivation (7 SDTM domains, ADSL/ADTTE, Define-XML v2.1, hand-rolled XPT writer), HL7 v2/CDISC LAB/FHIR parsers, BibTeX/RIS round-trip. All stdlib/typed, all unit-tested.
- **State machines in the schema layer** — screening-log CONSORT states, eCRF form lifecycle (draft → entered → signed → locked), randomisation allocation: monotonic transitions enforced in code, never by prompt.

7. Compliance by construction

Regulatory conformance is encoded as schemas and checklists the model must fill, not as tone-of-voice instructions:

- **PRISMA 2020** — all 42 items as structured fields; the flow diagram is generated SVG from real screening counts.
- **GRADE** — Summary-of-Findings with computed certainty (§5).
- **ICH E3** (CSR structure), **ICH E2A** (AE/SAE classification with a 24-hour SAE reporting clock), **CONSORT** (recruitment funnel codebook).
- **CDISC SDTM/ADaM/Define-XML exports**; **21 CFR Part 11** / **ALCOA+** — password re-authentication for e-signatures, study-lock gates, append-only audit trail with PI countersign, IQ/OQ/PQ validation pack with a 14-requirement traceability matrix.

8. FastAPI + SQLAlchemy practices

- **Routers per domain** (`web/ecrf.py` , `web/sr.py` , `web/portfolio.py` , ...), each thin: parse → authorize → service call → typed response. Business logic lives in `services/` , not endpoints.
- **Async end-to-end** — SQLAlchemy 2.0 async sessions (asyncpg in prod, aiosqlite in-memory for tests); a per-request session dependency; no sync I/O on the event loop (Docker SDK calls run in threads).

- **Additive migrations** (`_COLUMN_MIGRATIONS`) that work identically on SQLite and Postgres; idempotent `init_db` seeding. Alembic deferred until migrations outgrow this.
- **Schema-first persistence** — specialist output `model_dump_json()` is stored verbatim in `Message.final_answer` ; schemas must round-trip, so the DB is always re-renderable and machine-readable.
- **Typed everything** — mypy strict, ruff, pydantic-settings for config, 1,650+ tests against in-memory SQLite with env-var test controls.
- **Turn telemetry** — every turn emits a `done` event carrying `model_id` , token usage, per-tool call counts, and `route_rule` ; `cost_rollup.py` turns these into USD by workflow/model family. This event stream is the observability substrate (§10).

9. Security design

- **Authentication: AWS Cognito** (OIDC) — ALB-level auth in front plus app-level JWT validation (`auth/jwks.py` , `auth/tokens.py`); multi-tenant pool provisioning script; invite-based onboarding with a training/ delegation gate before clinical-write access.
- **Authorization: RBAC** (`auth/rbac.py`) — ~105 fine-grained permissions across 12 roles (admin, researcher, student, auditor, study_designer, PI, coordinator, data_manager, monitor, reviewer_1/2, adjudicator), with scope-typed grants (global / study / site / SR-review). Workflows themselves are permissioned (`skill.*`); the dispatcher enforces this and degrades implicit routes rather than leaking capability (§2).
- **Separation of duties** — SoD matrices in user-admin; dual-reviewer + adjudicator screening; PI code-break separated from allocation; monitors verify (SDV) what they cannot enter.
- **High-integrity actions** — password re-auth for signatures and study lock; append-only audit tables; PHI minimised in rollups (counts only).
- **Execution isolation** — the sandbox is the only place model-authored code runs: no network, resource-capped, ephemeral.
- **Known gaps (tracked, gated):**
- **TODO (T2 security gate):** `source_configs.api_key` is plaintext and returned verbatim by the admin GET — move to **AWS Secrets Manager** (or KMS envelope encryption) with write-only/masked API fields *before* any paywalled-source credentials are accepted on a public deployment.
- **TODO (T3):** production Cognito posture — sweep dev fallbacks (`default-user` placeholder), MFA/token-lifetime decisions.

10. Additional Production grade features

Item	Design intent
Quotas & spend limits (T1)	Hard USD budget from a trial start date across Bedrock + Tavily, enforced pre-flight per turn (429 + budget message, mirroring the existing daily token quota in <code>services/quota.py</code>); 80% warning banner; burn-rate on the portfolio dashboard.
Observability	Build on the <code>done</code> -event stream: per-route/latency/error dashboards, breaker-trip and <code>auth_fallback</code> alerting, structured log shipping (CloudWatch), trace IDs per turn.
SES integration (T4)	Domain identity + DKIM/SPF/DMARC, SES production access, Cognito invites via SES; <code>services/reminders.py</code> is already SES-ready behind <code>AWS_SES_FROM_EMAIL</code> .
Secrets manager (T2)	Credential encryption gate above; per-source usage limits (calls/day) on top of rate limits for publisher APIs (Elsevier/Lancet).
Domain + TLS (T5)	Route 53 + ACM cert on the ALB; Cognito callback URLs onto the real domain.

Maintenance: update alongside architectural changes; keep sections short and link to code rather than duplicating it.